MUHAMMAD FAIZAN YOUSAF

Interview Cheat Sheet

Automation Engineer — Brandspur

> *"I'm a final-year CS student at FAST-NUCES with hands-on experience in Python automation, REST API development, AI/ML systems, and full-stack development. I've built NLP pipelines, RAG systems, ETL workflows, and automated testing frameworks. This role directly matches what I've been building — automation + AI for real business problems."*

Python pandas · Flask · FastAPI · scikit-learn · PyTorch	**REST APIs** Built & consumed · auth · error handling
AI / ML RAG · NLP pipelines · LLM APIs · fine-tuning	**Databases** PostgreSQL · MySQL · MongoDB
Full Stack React · Node.js · Django · Next.js	**QA & Testing** Cypress · Playwright · Selenium · CI/CD

1 THE 5 INTERVIEW QUESTIONS

Q1 SQL — Top 3 Users by Revenue (3 Tables + Aggregation)

Join **users** → **orders** → **products**. Multiply quantity × price. Group by user. Get top 3.

```
■■ SQL ■■
SELECT u.user_id, u.name,
SUM(o.quantity * p.price) AS total_revenue
FROM users u
JOIN orders o ON u.user_id = o.user_id
JOIN products p ON o.product_id = p.product_id
GROUP BY u.user_id, u.name
ORDER BY total_revenue DESC
LIMIT 3;
```

- JOIN links tables on matching keys
- SUM(qty × price) = revenue per row
- GROUP BY → one row per user
- ORDER BY DESC + LIMIT 3 → top 3

Q2 Two Product — Find 2 numbers that multiply to target, return indices

```
■■ Python ■■
def two_product(nums, target):
seen = {} # value: index
for i, num in enumerate(nums):
if num == 0:
continue # skip zero to avoid division error
if target % num == 0: # must divide evenly
complement = target // num
if complement in seen:
return [seen[complement], i]
seen[num] = i
return []
# Test
print(two_product([2, 3, 4, 6], 12)) # [1, 2] → 3×4=12
```

- HashMap (dict) → O(n) optimal
- Check divisibility first (target % num == 0)
- Complement = target // num
- Skip zeros to avoid division errors

Q3 Extract Value from String (key passed as parameter)

Given 'order_id=123 user=ali total=500', extract value for a given key.

```
■■ Python ■■
def extract_value(text, key):
for part in text.split():
if part.startswith(key + '='):
return part.split('=')[1]
return None
# Test
text = 'order_id=123 user=ali total=500'
print(extract_value(text, 'total')) # 500
print(extract_value(text, 'user')) # ali
# Regex alternative (more robust)
import re
def extract_value_regex(text, key):
match = re.search(rf'{key}[=:\s]+(\w+)', text)
return match.group(1) if match else None
```

- split() breaks string into parts
- split('=')[1] gets value after '='
- startswith(key + '=') finds right part
- Regex version handles more formats

Q4 Return list from 1 to 15

```
■■ Python ■■
def generate_list():
return list(range(1, 16))
# Output
print(generate_list())
# [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

- range(1, 16) → 1 to 15 (16 is excluded)
- list() converts range object to actual list

Q5 FizzBuzz (1 to 15)

```
■■ Python ■■
def fizzbuzz(n):
result = []
for i in range(1, n + 1):
if i % 3 == 0 and i % 5 == 0: # CHECK BOTH FIRST!
result.append('FizzBuzz')
elif i % 3 == 0:
result.append('Fizz')
elif i % 5 == 0:
result.append('Buzz')
else:
result.append(i)
return result
print(fizzbuzz(15))
# [1, 2, 'Fizz', 4, 'Buzz', 'Fizz', 7, 8, 'Fizz', 'Buzz',
# 11, 'Fizz', 13, 14, 'FizzBuzz']
```

■ CRITICAL: Check BOTH (divisible by 3 AND 5) BEFORE checking either alone. 15 % 3 == 0 AND 15 % 5 == 0 → FizzBuzz. Most candidates get this wrong!

2 PYTHON QUICK REFERENCE

Lists & Loops

```
■■ Python ■■
# List comprehension
squares = [x**2 for x in range(10)]
# Enumerate — gives index + value
for i, val in enumerate(['a', 'b', 'c']):
print(i, val) # 0 a, 1 b, 2 c
# Filter — keep even numbers
evens = [x for x in range(20) if x % 2 == 0]
# Sort list of dicts by key (descending)
data = [{'name': 'ali', 'score': 90}, {'name': 'bob', 'score': 70}]
data.sort(key=lambda x: x['score'], reverse=True)
```

Dictionaries (HashMap)

```
■■ Python ■■
d = {}
d['key'] = 'value'
# Check if key exists
if 'key' in d:
print(d['key'])
# get() with default — safe lookup (no KeyError)
val = d.get('missing_key', 0)
# Count element frequencies
from collections import Counter
freq = Counter([1, 2, 2, 3, 3, 3]) # {3:3, 2:2, 1:1}
# defaultdict — auto-creates missing keys
from collections import defaultdict
dd = defaultdict(list)
dd['a'].append(1) # no KeyError even if 'a' didn't exist
```

Strings

```
■■ Python ■■
s = 'hello world'
s.split() # ['hello', 'world']
s.split(',') # split by comma
s.strip() # remove leading/trailing whitespace
s.replace('o', '0') # 'hell0 w0rld'
s.startswith('he') # True
s.find('world') # 6 (index of substring)
','.join(['a','b','c']) # 'a,b,c'
f'name={s}' # f-string formatting
```

Functions & Lambda

```
■■ Python ■■
def add(a, b=0): # default parameter
return a + b
# Lambda — anonymous one-liner function
double = lambda x: x * 2
# *args → captures extra positional args as tuple
# **kwargs → captures extra keyword args as dict
def flex(*args, **kwargs):
print(args) # tuple
print(kwargs) # dict
```

File I/O & Pandas

```
■■ Python ■■
# Read a text file
with open('file.txt', 'r') as f:
content = f.read()
# Write a text file
with open('file.txt', 'w') as f:
f.write('hello')
# Read/write CSV with pandas
import pandas as pd
df = pd.read_csv('data.csv')
df.to_csv('output.csv', index=False)
```

3 SQL QUICK REFERENCE

Core Query Structure

```sql
SELECT column1, column2, AGG_FUNC(col)
FROM table1
JOIN table2 ON table1.id = table2.fk_id
WHERE condition
GROUP BY column1, column2
HAVING AGG_FUNC(col) > value
ORDER BY column DESC
LIMIT n;
```

Aggregation Functions

```
SUM(col) -- total
AVG(col) -- average
COUNT(*) -- count all rows
COUNT(col) -- count non-null
MAX(col) -- maximum
MIN(col) -- minimum
```

JOIN Types

```
INNER JOIN -- matching rows in BOTH tables
LEFT JOIN -- all left + matching right
-- (NULL if no right match)
RIGHT JOIN -- all right + matching left
FULL JOIN -- all rows from both tables
```

Common Patterns

```sql
-- Top N by value
SELECT name, SUM(amount) AS total
FROM orders
GROUP BY name
ORDER BY total DESC
LIMIT 3;
-- Filter AFTER aggregation (use HAVING, not WHERE)
SELECT dept, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept
HAVING AVG(salary) > 50000;
-- Subquery — products above average price
SELECT *
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

4 DSA PATTERNS — LEETCODE EASY

HashMap / Dict Lookup — O(n)

```
# Two Sum (find two numbers that ADD to target)
def two_sum(nums, target):
seen = {}
for i, num in enumerate(nums):
comp = target - num
if comp in seen:
return [seen[comp], i]
seen[num] = i
return []
```

Sliding Window

```
# Max sum of k consecutive elements
def max_sum(arr, k):
window = sum(arr[:k])
max_s = window
for i in range(k, len(arr)):
window += arr[i] - arr[i - k] # slide right
max_s = max(max_s, window)
return max_s
```

Two Pointers

```
# Check if string is a palindrome
def is_palindrome(s):
l, r = 0, len(s) - 1
while l < r:
if s[l] != s[r]:
return False
l += 1
r -= 1
return True
```

Frequency Count

```
from collections import Counter
# Find most common element
def most_common(nums):
freq = Counter(nums)
return freq.most_common(1)[0][0]
# Check if two strings are anagrams
def is_anagram(s, t):
return Counter(s) == Counter(t)
```

Stack

```
# Validate bracket pairs: (){}[]
def is_valid(s):
stack = []
mapping = {')': '(', '}': '{', ']': '['}
for char in s:
if char in mapping:
top = stack.pop() if stack else '#'
if mapping[char] != top:
return False
else:
stack.append(char)
return not stack
```

Time Complexity Cheat Sheet

Operation	Complexity
List access by index	**O(1)**
List append	**O(1)**
List search (in)	**O(n)**
Dict get / set / in	**O(1)**
Sorting	**O(n log n)**
Nested loops	**O(n²)**
Binary search	**O(log n)**

5 OOP — OBJECT ORIENTED PROGRAMMING

4 Pillars — Know These Cold

Encapsulation	Hide data inside class, expose only what's needed (private variables).
Inheritance	Child class inherits properties and methods from parent class.
Polymorphism	Same method name, different behavior in different subclasses.
Abstraction	Hide complex logic behind a simple interface.

```
■■ Python ■■
class Animal:
def __init__(self, name):
self.name = name # public attribute
self.__age = 0 # private (Encapsulation — double underscore)
def speak(self): # method to be overridden
return '...'
def get_age(self): # getter for private attribute
return self.__age
class Dog(Animal): # Inheritance — Dog extends Animal
def speak(self): # Polymorphism — same name, different behavior
return f'{self.name} says Woof!'
class Cat(Animal):
def speak(self):
return f'{self.name} says Meow!'
# Polymorphism in action — same loop, different outputs
animals = [Dog('Rex'), Cat('Whiskers')]
for a in animals:
print(a.speak())
# Rex says Woof!
# Whiskers says Meow!
```

Key OOP Terms

`__init__`	Constructor — runs automatically when object is created
`self`	Reference to the current instance of the class
`super()`	Call a method from the parent class
`@classmethod`	Method on the class itself, not an instance (cls instead of self)
`@staticmethod`	Utility method — no self or cls, like a regular function inside a class
`@property`	Access a method as if it were an attribute (no parentheses needed)

6 REST APIS & INTEGRATION

HTTP Methods & Status Codes

Method	Purpose	Code	Meaning
GET	Read data	200	OK
POST	Create data	201	Created
PUT	Replace data	400	Bad Request
PATCH	Partial update	401	Unauthorized
DELETE	Remove data	404	Not Found
		500	Server Error

Calling APIs with Python (requests)

```python
import requests
# GET request with auth header and query params
response = requests.get(
'https://api.example.com/orders',
headers={'Authorization': 'Bearer TOKEN'},
params={'status': 'pending'}
)
if response.status_code == 200:
data = response.json()
else:
print(f'Error: {response.status_code}')
# POST request with JSON body
payload = {'order_id': '123', 'status': 'shipped'}
r = requests.post(
'https://api.example.com/update',
json=payload,
headers={'Authorization': 'Bearer TOKEN'}
)
```

Simple Flask API

```python
from flask import Flask, request, jsonify
app = Flask(__name__)
inventory = {'apple': 100, 'banana': 50}
@app.route('/inventory', methods=['GET'])
def get_inventory():
return jsonify(inventory)
@app.route('/inventory/', methods=['GET'])
def get_item(item):
if item in inventory:
return jsonify({'item': item, 'qty': inventory[item]})
return jsonify({'error': 'not found'}), 404
if __name__ == '__main__':
app.run(debug=True)
```

7 AI / ML & LLM INTEGRATION

Calling Claude API

```python
import anthropic
client = anthropic.Anthropic(api_key='YOUR_KEY')
message = client.messages.create(
model='claude-opus-4-5',
max_tokens=1024,
messages=[
{'role': 'user', 'content': 'What is the stock level for product ABC?'}
]
)
print(message.content[0].text)
```

NLP — Email Classification (Zero-Shot)

```python
from transformers import pipeline
# Zero-shot: no training data needed, just provide labels
classifier = pipeline('zero-shot-classification')
email = 'Where is my order #12345? It has been 2 weeks!'
labels = ['order inquiry', 'complaint', 'general inquiry']
result = classifier(email, labels)
print(result['labels'][0]) # complaint
```

Key ML Terms

MAPE	Mean Absolute Percentage Error — measures forecast accuracy
Moving Average	Average of last N data points — used in demand forecasting
Exponential Smoothing	Weighted average where recent data gets higher weight
RAG	Retrieval Augmented Generation — your FYP uses this
Fine-tuning	Training a pretrained model further on your specific dataset
Embeddings	Convert text to numeric vectors for similarity search

8 PANDAS — SUPPLY CHAIN DATA

```python
■■ Python ■■
import pandas as pd
df = pd.read_csv('inventory.csv')
# Basic exploration
df.head() # first 5 rows
df.shape # (rows, cols) tuple
df.columns # column names
df.dtypes # data type of each column
df.describe() # count, mean, std, min, max
df.isnull().sum() # count of null values per column
# Filter rows
low_stock = df[df['quantity'] < 10]
expensive = df[df['price'] > 100]
# Group and aggregate
revenue = df.groupby('category')['revenue'].sum()
top3 = revenue.nlargest(3)
# Merge — equivalent to SQL JOIN
merged = pd.merge(orders, products, on='product_id', how='inner')
# Add a calculated column
df['total_value'] = df['quantity'] * df['price']
# Sort and get top 10
df.sort_values('revenue', ascending=False).head(10)
```

9 DBMS CONCEPTS

ACID Properties

Atomicity	All operations in a transaction succeed, or none do — no partial commits.
Consistency	Database remains in a valid state before and after every transaction.
Isolation	Concurrent transactions do not interfere with each other.
Durability	Committed data survives crashes (written to disk).

Normalization

1NF	No repeating groups — every cell holds a single atomic value.
2NF	1NF + no partial dependency on a composite primary key.
3NF	2NF + no transitive dependency (non-key column depending on another non-key column).

SQL in Python (sqlite3)

```
■■ Python ■■
import sqlite3
conn = sqlite3.connect(':memory:') # or 'mydb.db' for a file
cursor = conn.cursor()
cursor.execute('''
CREATE TABLE users (
id INTEGER PRIMARY KEY,
name TEXT,
revenue REAL
)
''')
cursor.execute("INSERT INTO users VALUES (1, 'Ali', 5000)")
conn.commit()
rows = cursor.execute('SELECT * FROM users').fetchall()
print(rows) # [(1, 'Ali', 5000.0)]
conn.close()
```

10 SUPPLY CHAIN BASICS

EOQ	Economic Order Quantity — optimal order size to minimize holding + ordering cost.
Safety Stock	Buffer inventory kept to prevent stockouts during demand spikes or delays.
Reorder Point	Inventory level that triggers placing a new purchase order.
ABC Analysis	A = high value items, B = medium value, C = low value. Prioritize A.
FIFO / LIFO	First In First Out / Last In First Out — inventory costing methods.
KPI	Key Performance Indicator — measurable metrics to track performance.
3PL	Third Party Logistics — outsourced shipping, warehousing, and fulfilment.
ERP	Enterprise Resource Planning — integrated software (SAP, NetSuite, Odoo).

Reorder Point Formula

```python
# Reorder Point = (Daily Sales × Lead Time) + Safety Stock
daily_sales = 50 # units sold per day
lead_time = 7 # days until order arrives
safety_stock = 100 # buffer to cover unexpected demand
reorder_point = (daily_sales * lead_time) + safety_stock
print(f'Reorder at: {reorder_point} units') # 450
```

11 YOUR PROJECTS — WHAT TO SAY

HealthMate (FYP)

Stack: React Native · Flask · MySQL · XGBoost · Docker

- Mobile health app with real-time risk prediction from wearable sensor data.
- Built REST API backend with JWT authentication and role-based access control.
- Integrated XGBoost model for health risk scoring — response time under 5 seconds.

RAG Clinical Notes

Stack: Python · FAISS · Sentence Transformers · Streamlit

- Built a Retrieval Augmented Generation system over medical records.
- Used vector embeddings + cosine similarity to retrieve relevant documents.
- Direct match to the JD requirement for LLM / NLP pipelines.

RentEase

Stack: Next.js · Django · PostgreSQL · FastAPI

- Full-stack SaaS rental management platform.
- Django REST backend with PostgreSQL, FastAPI microservice for PDF generation.
- Demonstrates real-world REST API build and consume experience.

QLoRA Fine-Tuning

Stack: Qwen2-VL · HuggingFace · Gradio

- Fine-tuned a vision-language model with 4-bit quantization.
- ROUGE-L score improved from 0.18 → 0.63 (3.5× improvement).
- Only trained 3% of parameters using LoRA — efficient and practical.

Node.js CI/CD Pipeline

Stack: Mocha · Chai · Jenkins · Page Object Model

- Built automated test framework with 20+ test cases.
- Configured Jenkins CI pipeline with HTML reporting.
- Shows DevOps and automation engineering mindset.

Speech-to-Text Pipeline (Internship)

Stack: Whisper · FAISS · NLP

- Built meeting transcription and summarization pipeline.
- Reduced manual review effort by approximately 60%.
- FAISS semantic search for fast retrieval across transcripts.

12 DEBUGGING — WHEN YOU SEE BROKEN CODE

1. Read the error message — it gives you the line number and error type.
2. Check variable types — print(type(x)) is your best friend.
3. Add print() statements just before the line that errors.
4. Check for off-by-one errors in loops — range(n) vs range(n+1).
5. Check for None — did the function return nothing when you expected a value?
6. Check indentation — Python is whitespace-sensitive; one wrong indent breaks everything.

Common Python Bugs

```
■■ Python ■■
# Bug 1 — Off-by-one
for i in range(len(arr)): # correct
for i in range(len(arr) + 1): # IndexError on last iteration
# Bug 2 — Mutating a list while iterating over it (items get skipped)
# WRONG:
for item in my_list:
my_list.remove(item)
# RIGHT — build a new list instead:
my_list = [x for x in my_list if condition]
# Bug 3 — Integer vs float division
10 / 3 # 3.333 (true division — float result)
10 // 3 # 3 (floor division — integer result)
# Bug 4 — None comparison
if x == None: # works but not Pythonic
if x is None: # correct way
# Bug 5 — Shallow copy vs deep copy
import copy
shallow = original.copy() # inner objects still shared!
deep = copy.deepcopy(original) # fully independent copy
```

13 JD REQUIREMENTS VS YOUR EXPERIENCE

JD Requirement	Your Experience	Strength
Python (pandas, Flask, FastAPI)	HealthMate, RentEase, ML projects	★★★★
REST API build & consume	RentEase FastAPI, Flask APIs, internship	★★★★
SQL proficiency	PostgreSQL, MySQL across 5+ projects	★★★★
LLM API integration	RAG system, Claude/OpenAI aware	★★★
NLP pipelines	Email classifier, Whisper pipeline, RAG	★★★★
Zapier / Make	Limited — be honest, say you're learning	★★
Google Apps Script	Limited — be honest	★★
Supply chain concepts	Basic — studied for this interview	★★
ML forecasting	scikit-learn, XGBoost, statsmodels	★★★

Questions to Ask Them (ask 1-2)

→ What does a typical client automation project look like end-to-end?

→ Which ERP systems do most of your clients use — NetSuite, Odoo, or SAP?

→ What is the team structure — do engineers own full projects or specialize?

→ What tech stack is most used for the Python automations currently?

✓ You have solid projects. Talk about them with confidence. Good luck! ■